

Programming Notes

1. What is Programming?

Programming is the process of giving instructions to a computer to perform specific tasks using a programming language. A program is a set of instructions written by a programmer.

Example Program

```
print("Hello World")
```

2. Types of Programming Languages

Programming languages are mainly divided into two categories: Low-Level Languages and High-Level Languages.

a) Low-Level Language

Low-level languages are close to computer hardware and difficult for humans to understand. They provide faster execution.

Examples of Low-Level Languages

- Machine Language
- Assembly Language

b) High-Level Language

High-level languages are easy to read, write, and understand by humans.

Examples of High-Level Languages

- Python
- Java
- C++
- JavaScript

3. Generations of Programming Languages

4. Features of Programming Languages

- Simplicity
- Portability
- Efficiency
- Readability
- Flexibility
- Reusability

5. Programming Paradigms

Programming paradigms are different approaches to writing programs.

a) Procedural Programming

Program is divided into functions or procedures.

Examples: C, Pascal

b) Object-Oriented Programming (OOP)

Based on objects and classes.

Examples: Python, Java, C++

c) Functional Programming

Uses functions and immutability.

Examples: Haskell, Scala

d) Logic Programming

Based on logic and rules.

Example: Prolog

6. Steps in Program Development

Problem Definition → Algorithm → Flowchart → Coding → Testing → Debugging → Documentation → Maintenance

7. Algorithm

An algorithm is a step-by-step procedure to solve a problem.

Characteristics of Algorithm

- Finite
- Clear
- Efficient
- Logical

Example Algorithm for Addition

1. Start
2. Input A and B
3. Sum = A + B
4. Display Sum
5. Stop

8. Flowchart

A flowchart is a graphical representation of an algorithm.

Common Flowchart Symbols

- Oval ☐ Start/Stop
- Rectangle ☐ Process
- Parallelogram ☐ Input/Output
- Diamond ☐ Decision

9. Compiler vs Interpreter

A compiler converts the entire program into machine code at once, while an interpreter converts code line by line.

Compiler	Interpreter
Converts whole program at once	Converts line by line
Faster execution	Slower execution
Shows all errors together	Shows one error at a time

Generation	Description	Examples
1GL	Machine Language	Binary Code
2GL	Assembly Language	Assembly
3GL	High-Level Language	Python, C
4GL	Query Languages	SQL
5GL	AI Languages	Prolog